

武汉理工大学

企业级应用软件设计与开发课程报告

Agentic AI 原生开发

课程名称	企业级应用软件设计与开发
项目名称	MiniAgent_QQBot
方 向	Agentic AI 原生开发
学 号	2025303055
姓 名	郑军超
专 业	软件工程
指导老师	戚欣
提交日期	2026 年 6 月 22 日

目录

一、 选题背景与设计思想	4
1.1 选题背景	4
1.2 问题定义	4
1.3 现有方案不足	5
1.4 项目价值	6
1.5 技术路线	6
二、 Specs 规格文档	7
2.1 Product Spec	7
2.2 Architecture Spec	8
2.3 API Spec	9
三、 系统架构与设计	10
3.1 核心架构图	10
3.2 Agent 交互流程	11
3.3 数据流设计	12
四、 关键实现与代码展示	14
4.1 QQBot 消息接入实现	14
4.2 工具定义	16
4.3 TurnIntent 意图识别实现	18
4.4 Skill 配置与执行实现	20
4.5 MiniAgentApp 主流程实现	23
4.6 Agent 核心循环	25
4.7 RuntimeVerifier 与 RuntimeRecovery 实现	27
4.8 Harness 系统实现	30
4.9 配置文件	31
4.4 AI IDE 使用截图	33
五、 测试与评估	35

5.1 功能测试	35
5.2 Agent 行为评估	37
5.3 Benchmark	40
5.4 Demo 截图	42
六、 系统升级与扩展	44
6.1 可扩展架构	44
6.2 下一阶段计划	44
6.3 AI 能力演进路径	45
七、 课程总结	45
7.1 个人收获	45
7.2 工程思维转变	46
7.3 对课程的建议	47

一、选题背景与设计思想

1.1 选题背景

随着大语言模型的发展，智能体系统逐渐从传统的文本问答工具发展为能够理解用户意图、调用外部工具并完成实际任务的任务型 Agent。与此同时，QQ 作为常见的即时通信平台，具有较强的用户交互便利性。用户在 QQ 聊天场景中不仅会提出普通问答需求，还可能希望机器人能够完成文件总结、文档转换、代码搜索、天气查询等实际任务。因此，将大语言模型 Agent 能力引入 QQBot 场景，能够使传统 QQBot 从固定命令式机器人扩展为具备自然语言理解和工具调用能力的智能助手。

传统 QQBot 通常依赖关键词匹配或固定命令触发，例如用户需要输入特定格式的命令才能获得结果。这种方式虽然实现简单，但灵活性较弱，难以处理用户自然语言表达下的复杂任务。例如，当用户发送“帮我总结一下这个 PDF”时，传统 QQBot 很难自动判断该请求属于文件问答任务，也无法主动读取用户上传的 PDF 文件并基于文件内容生成总结。再如，当用户发送“把这个 Excel 导出成 PDF”时，传统 QQBot 也难以自动判断这是文件转换任务，并调用相应的 Excel 处理工具完成真实文件生成。

基于上述背景，本项目设计并实现了一个面向 QQ 聊天场景的任务型智能 Agent。该系统以 QQBot 作为用户交互入口，以大语言模型作为任务理解和决策核心，通过工具调用和 Skill 模块完成文件读取、文件处理、代码导航、天气查询等任务。与传统 QQBot 相比，本项目中的 Agent 不再局限于固定命令响应，而是能够根据用户的自然语言请求自主判断任务类型，并选择合适的工具完成操作。

1.2 问题定义

本项目需要解决的核心问题是：如何设计一个基于 QQBot 的任务型智能 Agent，使其能够接收用户在 QQ 中发送的文本消息和上传文件，理解用户当前任务需求，选择合适的工具或 Skill 执行操作，并将执行结果返回给用户。

围绕这一核心问题，系统需要解决若干具体子问题。首先，QQBot 需要能够接收用户文本消息和上传文件，并将不同形式的 QQ 消息统一转换为 Agent 内部可处理的消息格式。其次，Agent 需要能够判断用户当前请求的任务类型，例如普通问答、文件问答、文件转换、代码搜索或天气查询等。再次，Agent 需要能够判断当前任务是否依赖用户上传的文件，若任务涉及文件内容，则必须读取文件后再进行回答；若任务要求生成新文件，则需要调用相应的文件处理工具并保存输出结果。此外，Agent 还需要根据任务类型选择不同工具或 Skill，例如 PDF 工具、Word 工具、Excel 工具、代码搜索工具和天气查询工具等。最后，系统

需要将工具执行结果、生成文件路径或文本回答通过 QQBot 返回给用户，并记录 Agent 的执行过程，以便后续测试、调试和评估。

因此，本项目并不是实现一个简单的 QQ 聊天机器人，而是设计一个能够在 QQ 聊天场景中完成真实任务的智能 Agent。该 Agent 需要具备消息接入、意图识别、文件管理、工具调用、结果返回和运行记录等完整能力。

1.3 现有方案不足

在传统 QQBot 设计中，机器人通常根据固定命令或关键词进行响应。例如，用户输入天气命令后，系统调用天气接口返回结果；用户输入帮助命令后，系统返回固定菜单。这种方式适用于简单、明确、规则化的任务，但难以适应自然语言表达下的复杂需求。

首先，传统 QQBot 缺少对自然语言任务的理解能力。用户在真实聊天场景中往往不会按照固定命令输入，而是使用更加自然和模糊的表达方式。例如，“帮我看看这个文档主要讲了什么”“把这个表格处理一下并导出 PDF”“找一下这个项目里核心循环的实现位置”等请求都需要系统理解上下文和任务目标，而不是简单地匹配关键词。

其次，传统 QQBot 缺少对上传文件的上下文管理能力。用户可能先上传文件，然后在下一条消息中使用“这个文件”“刚才的表格”“这个 PDF”等指代表达。系统需要将用户上传的文件与当前会话关联起来，才能正确理解用户请求。如果没有附件管理和会话管理机制，机器人很难知道用户所说的“这个”具体指哪个文件。

再次，传统 QQBot 缺少工具调用和任务执行能力。对于文件总结、PDF 合并、Excel 导出 PDF、Word 修订处理等任务，仅靠自然语言回复无法真正满足用户需求。用户期望系统直接完成操作，而不仅仅是告诉用户应该如何操作。因此，Agent 必须能够调用外部工具或脚本，对文件进行真实处理，并生成可用的输出结果。

最后，传统 QQBot 通常缺少运行过程记录和测试评估机制。当机器人回复错误或任务失败时，开发者难以判断问题出现在意图识别、工具选择、工具执行还是结果返回环节。对于课程项目而言，仅展示成功运行截图并不足以说明系统设计的完整性，因此需要记录 Agent 的执行过程，并通过测试和 Benchmark 对系统能力进行评估。

1.4 项目价值

本项目的价值在于将传统 QQBot 从固定命令式机器人扩展为一个任务型智能 Agent。用户不再需要按照固定格式输入指令，而是可以直接在 QQ 中通过自然语言提出需求，例如文件总结、文档转换、代码搜索和天气查询等。系统能够接收用户文本和上传文件，理解当前任务意图，并调用相应工具或 Skill 完成真实操作，从而提升 QQBot 在实际聊天场景中的智能化程度和实用性。

同时，本项目也体现了 Agentic AI 的工程设计价值。系统不仅使用大语言模型生成回复，还围绕 QQBot Agent 设计了消息接入、会话管理、附件管理、意图识别、工具调用、Skill 扩展、运行检查、失败恢复和 Trace 记录等模块，使 Agent 具备较完整的任务执行闭环。通过这些设计，系统能够更稳定地处理真实文件任务，也便于后续测试、调试、评估和功能扩展。

1.5 技术路线

本项目的技术路线可以概括为“QQBot 接入—消息处理—意图识别—工具选择—任务执行—结果验证—回复返回—过程记录”。

设计 Channel -> MessageBus -> MiniAgentApp -> AgentLoop -> ToolRegistry -> RuntimeVerifier 的 Agent Runtime 架构，将用户消息、会话历史、附件文件、长期记忆、Skill 路由和工具调用统一到可追踪执行链路中。

实现 RuntimeVerifier 与 RuntimeRecovery，对文件 grounding、输出文件生成、脚本执行成功、虚假路径声明和 progress-only 回复进行运行时校验与自动恢复，降低 Agent “假装读文件 / 假装保存 / 假装执行脚本”的风险。

设计项目级 Skill 系统，支持 SKILL.md + actions.json + scripts/ 的能力包结构，通过 Skill Router 自动选择，并统一通过 run_skill_script 执行脚本。

实现结构化 Memory 系统，实现长期记忆模块，区分短期 session、历史摘要和结构化长期记忆 memory_store.jsonl，支持 embedding 检索、关键词召回、记忆去重与冲突处理。

构建 Harness 评测与调试体系，支持 isolated eval、runtime trace、deterministic replay 和 regression compare，用于复盘 Agent 每轮的 llm_request -> tool_call -> tool_result -> violation/recovery -> turn_completed 执行链路。

因此，本项目的整体技术路线体现了任务型 Agent 的完整执行闭环：用户提出任务，Agent 理解任务，工具完成任务，系统检查结果，最后将可靠结果返回用户。

二、Specs 规格文档

2.1 Product Spec

本项目设计的是一个面向 QQ 聊天场景的任务型智能 Agent。与传统 QQBot 主要依赖固定命令或关键词触发不同，本项目希望通过大语言模型增强 QQBot 的自然语言理解能力，使用户能够直接在 QQ 中通过自然语言提出任务请求，并由 Agent 自动完成文件读取、文件处理、代码搜索、天气查询等操作。因此，本项目的产品目标不是实现一个简单的聊天机器人，而是构建一个能够接收 QQ 消息、理解任务意图、调用工具执行操作并返回真实结果的智能体系统。

从使用场景来看，该 QQBot Agent 需要支持用户在聊天过程中发送文本消息和上传文件。用户既可以提出普通问答请求，也可以提出依赖文件的任务请求。例如，当用户上传 PDF 后发送“帮我总结这个文件”时，系统需要识别该请求属于文件问答任务，并读取上传文件内容后再生成总结；当用户上传 Excel 后发送“把这个表格导出成 PDF”时，系统需要识别该请求属于文件转换任务，并调用对应的 Excel 处理 Skill 生成真实 PDF 文件。由此可见，本项目中的 Agent 不只是生成自然语言回复，还需要具备对外部工具和文件资源的调度能力。

在功能需求方面，系统需要满足多渠道消息接入、会话管理、附件管理、意图识别、工具调用、Skill 执行、结果返回和过程记录等要求。多渠道消息接入保证 QQBot 与本地 CLI 测试入口能够进入同一套 Agent 处理流程；会话管理用于保存用户多轮交互历史，使 Agent 能够理解“这个文件”“刚才上传的表格”等上下文指代；附件管理用于保存和定位用户上传文件；意图识别用于判断当前任务是否需要文件证据、是否需要生成输出文件、是否需要调用脚本；工具调用和 Skill 执行则负责完成真实任务；过程记录用于支持后续测试、调试和评估。

为了保证 Agent 的任务执行具有明确边界，系统还需要定义基本验收标准。对于文件问答任务，Agent 必须基于真实文件读取结果生成回答；对于文件生成任务，Agent 必须生成真实输出产物后才能返回完成结果；对于脚本型 Skill 任务，Agent 必须通过统一工具执行受控脚本，而不能仅凭自然语言声明任务完成。通过这些需求规格，QQBot Agent 的功能目标被明确为“理解用户请求并完成真实任务”，而不是简单地“回复用户消息”。具体 Product Spec 见：

<https://github.com/dwjds/cs599-project/blob/main/docs/specs/product-spec.md>

2.2 Architecture Spec

根据项目需求，系统整体采用分层式架构设计。最外层是 QQBot 交互入口，负责接收用户在 QQ 中发送的文本消息和文件消息，并将 Agent 的最终结果返回给用户。为了降低平台耦合，QQBot Channel 不直接实现 Agent 逻辑，而是将平台消息转换为系统内部统一的消息格式，再交由 MessageBus 和 MiniAgentApp 处理。这样设计能够使 QQBot、CLI 或未来其他平台共用同一套 Agent 执行流程。

系统的核心控制模块是 MiniAgentApp。它负责组织一次用户请求从接收到回复的完整流程，包括保存附件、加载会话历史、进行意图识别、选择相关 Skill、构造大语言模型上下文、执行工具调用循环、检查任务结果并生成最终回复。在该流程中，TurnIntent 模块承担任务理解功能，它将用户自然语言请求转化为结构化任务信息，例如当前任务是否为文件问答、是否需要输出文件、源文件格式是什么、目标格式是什么、是否需要脚本执行等。TurnIntent 的输出决定后续 Agent 是否需要读取文件、调用 Skill 或进入工具循环。

工具执行层由 ToolRegistry 和 SkillRuntime 构成。ToolRegistry 负责统一注册和调用系统工具，包括读取文件、写入文件、查找文件、搜索代码、读取上传附件、保存输出文件以及执行 Skill 脚本等。SkillRuntime 则负责执行具体 Skill 中定义的脚本，例如 PDF 合并、PDF 抽页、Excel 导出 PDF、Word 修订处理等。通过该层设计，LLM 不直接操作系统资源，而是通过受控工具接口完成真实任务，从而提高系统执行过程的可管理性。

系统还设计了运行检查与恢复机制。RuntimeVerifier 用于检查 Agent 的最终回复是否与实际执行结果一致，例如文件问答是否读取了文件、文件转换是否生成了输出产物、脚本任务是否真正执行成功。RuntimeRecovery 用于在验证失败或工具执行失败时生成恢复策略，引导 Agent 重新读取文件、重新调用脚本或返回可信失败说明。TraceSink 则记录本轮任务中的意图识别、工具调用、工具返回、产物生成、失败恢复和最终回复等事件，为后续 Benchmark、Replay 和 Regression 提供依据。

因此，本项目的架构设计体现了任务型 Agent 的完整执行闭环：QQBot 负责用户交互，MiniAgentApp 负责流程编排，TurnIntent 负责任务理解，ToolRegistry 和 SkillRuntime 负责真实执行，RuntimeVerifier 和 RuntimeRecovery 负责执行检查与失败处理，Trace 负责运行过程记录。具体 Architecture Spec 见：

<https://github.com/dwjds/cs599-project/blob/main/docs/specs/architecture-spec.md>

2.3 API Spec

为了使 QQBot Agent 的不同模块能够稳定协作，系统需要定义统一的核心接口与数据结构。首先是消息接口。用户从 QQBot 或 CLI 发出的请求会被封装为 `InboundMessage`，其中包含消息来源、用户标识、会话标识、文本内容和附件信息。系统返回给用户的结果则可以抽象为 `OutboundMessage`，其中包含目标通道、回复文本、输出文件或其他元数据。通过统一消息接口，系统可以屏蔽不同平台的差异，使 QQBot 和 CLI 共享同一套 Agent 处理逻辑。

其次是任务意图接口。`TurnIntent` 是连接自然语言请求和系统执行流程的关键结构。它通常包括 `operation`、`requires_file_grounding`、`requires_output_file`、`requires_script`、`source_format`、`target_format`、`confidence` 和 `reasons` 等字段。`operation` 表示当前任务类型，`requires_file_grounding` 表示是否需要读取文件证据，`requires_output_file` 表示是否需要生成输出产物，`requires_script` 表示是否需要执行脚本，`source_format` 和 `target_format` 分别表示源文件格式和目标格式。通过该结构，系统能够把用户自然语言请求转化为可执行的任务控制信号。

再次是工具接口。每个工具需要提供工具名称、工具描述、参数结构和执行方法。大语言模型通过工具定义了解当前可用能力，并在需要时发起工具调用。工具执行后返回 `ToolResult`，其中包含执行是否成功、输出内容、错误信息和可能产生的文件产物。对于脚本型 `Skill`，系统通过 `run_skill_script` 统一执行，并要求脚本路径位于对应 `Skill` 的 `scripts` 目录下，避免模型直接拼接任意命令或访问不受控路径。

此外，系统还需要定义验证、恢复和 `Trace` 接口。验证接口用于判断最终回复是否满足任务要求，例如是否存在文件读取证据、是否存在输出产物、是否存在未执行脚本却声称完成的情况。恢复接口根据验证失败原因生成恢复策略，例如要求重新读取上传文件、重新执行 `Skill` 脚本或返回明确错误说明。`TraceEvent` 则用于记录 Agent 执行过程中的关键事件，包括 `turn_intent`、`llm_request`、`llm_response`、`tool_call`、`tool_result`、`evidence_collected`、`output_artifact`、`recovery_plan` 和 `turn_completed` 等。通过这些事件，开发者可以追踪 Agent 每一步行为，并对系统进行测试和回归分析。

这些核心接口与数据结构保证了 QQBot Agent 的各模块之间具有明确的交互协议，使系统能够从用户消息接入、任务理解、工具执行、结果验证到最终回复形成完整链路。具体 API Spec 见：

<https://github.com/dwjds/cs599-project/blob/main/docs/specs/api-spec.md>

三、系统架构与设计

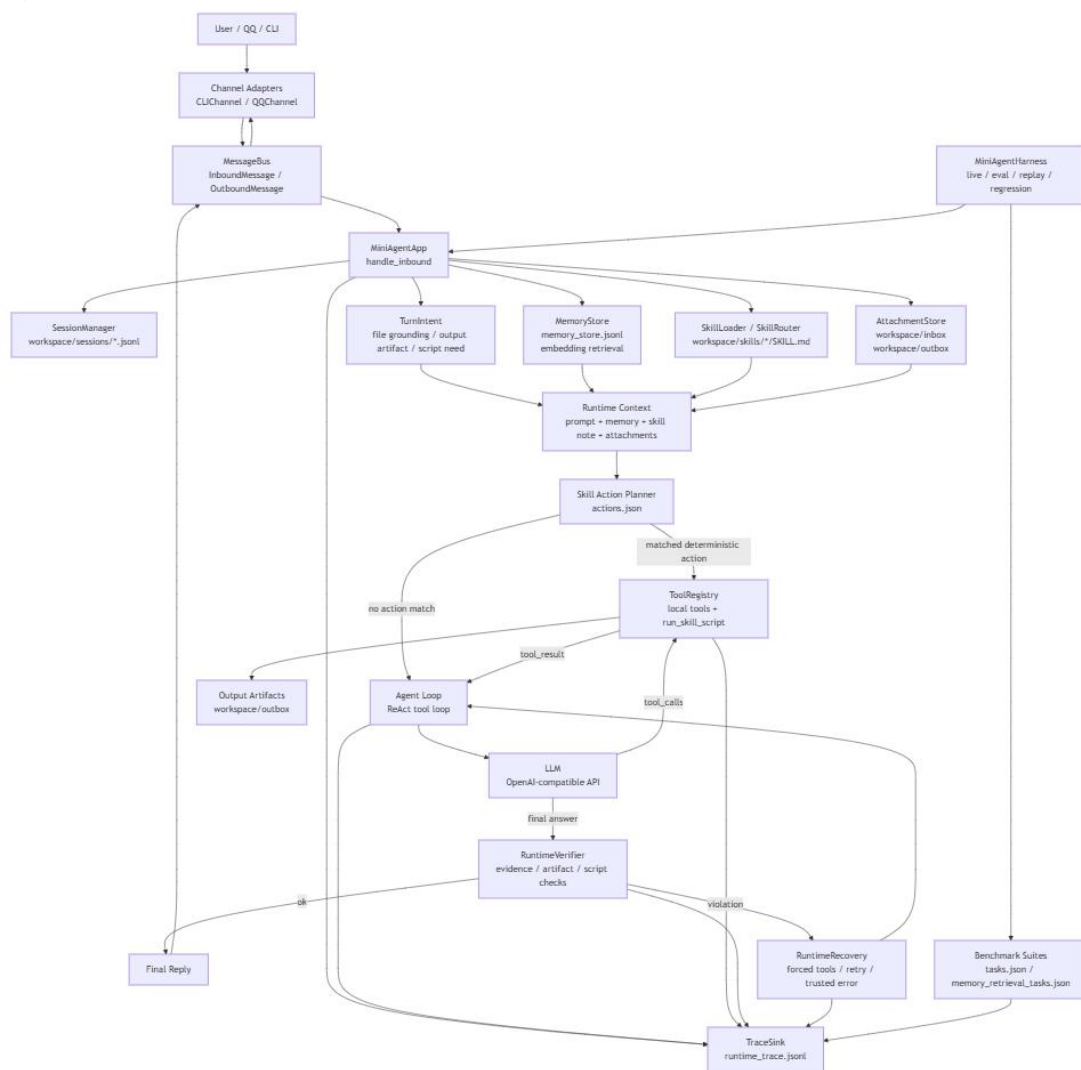
3.1 核心架构图

本项目采用分层式架构设计。系统最外层是 QQBot 交互入口，负责与用户进行自然语言对话和文件传输。QQBot 接收到用户消息后，会将消息转化为统一的内部格式，并通过 MessageBus 传递给 MiniAgentApp。MiniAgentApp 是整个系统的核心控制模块，负责组织会话管理、附件处理、意图识别、Skill 选择、工具调用、结果验证和最终回复。

在 MiniAgentApp 内部，系统首先根据用户消息和附件信息构造当前任务上下文。随后，TurnIntent 模块判断当前任务类型，并生成结构化任务描述。对于文件相关任务，系统会读取用户上传文件，并将文件内容作为 Agent 后续回答或处理的依据。对于需要生成输出文件的任务，系统会选择对应 Skill，并通过 ActionPlanner 或 Agent Loop 调用脚本完成操作。

工具执行部分由 ToolRegistry 和 SkillRuntime 支撑。ToolRegistry 为 Agent 提供统一工具接口，避免 Agent 直接操作底层文件系统或脚本。SkillRuntime 则负责执行具体 Skill 中定义的脚本，例如 PDF 合并、Excel 导出 PDF 或 Word 修订处理。执行结果会返回给 MiniAgentApp，并参与最终回复生成。

系统最后通过 RuntimeVerifier 检查任务是否完成。例如，文件总结任务需要检查是否读取了文件，文件转换任务需要检查是否生成了输出文件。如果检查通过，系统将结果返回给 QQ 用户；如果检查失败，则由 RuntimeRecovery 生成恢复策略。整个过程会被 TraceSink 记录，便于开发者后续分析和测试。



3.2 Agent 交互流程

一次完整的 Agent 交互流程从 QQ 用户发送消息开始。用户可以只发送文本消息，也可以先上传文件再发送任务请求。QQBot Channel 接收到消息后，将其封装为 InboundMessage，并通过 MessageBus 传递给 MiniAgentApp。

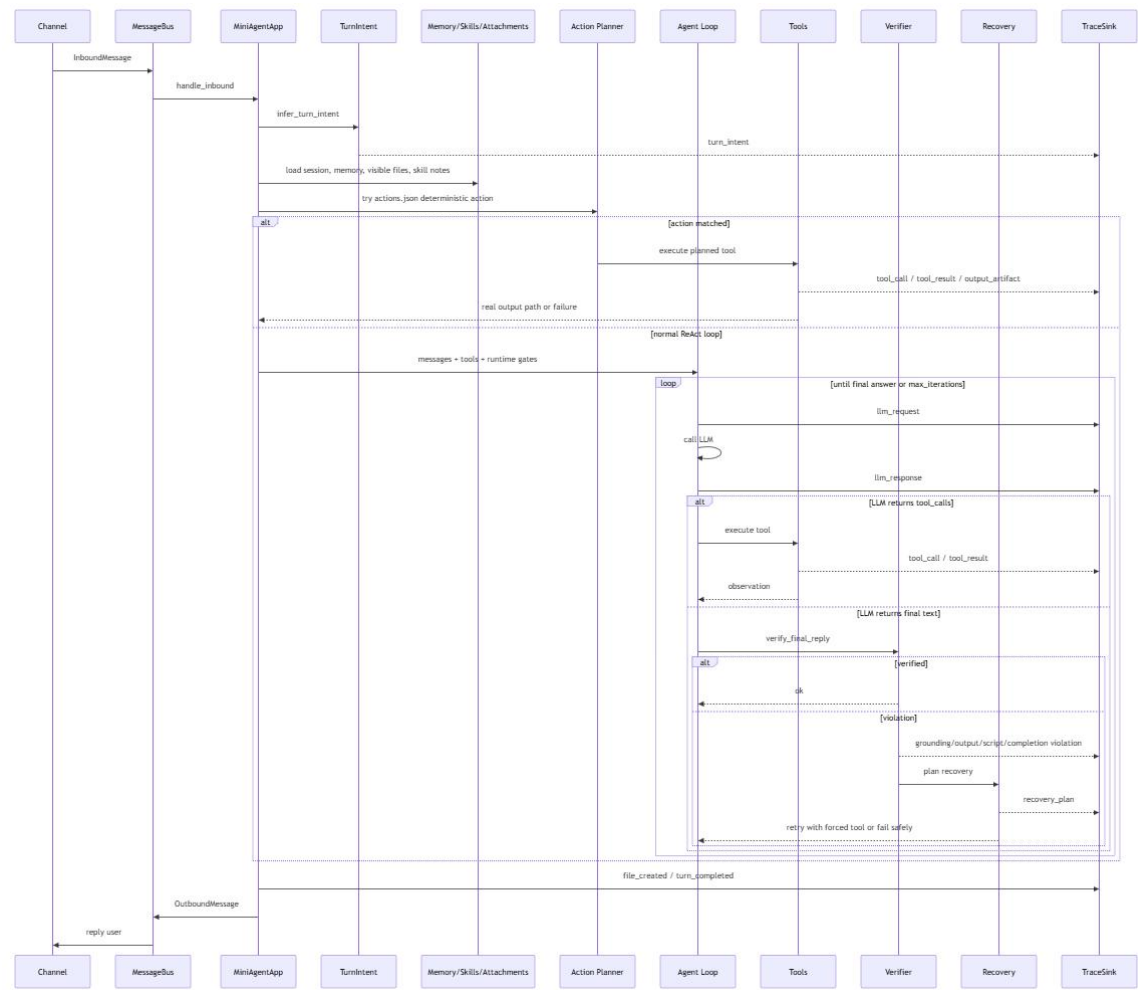
MiniAgentApp 首先保存用户上传的附件，并读取当前会话历史。如果用户当前请求涉及前文中的文件或上下文，系统可以通过会话记录找到相关信息。然后，TurnIntent 模块对用户请求进行分析，判断该请求属于何种任务类型。

如果用户请求是文件问答任务，例如“帮我总结这个 PDF”，系统会读取上传文件内容，并将文件内容加入 LLM 上下文。LLM 随后基于文件内容生成总结，最终通过 QQBot 返回用户。如果用户请求是文件转换任务，例如“把这个 Excel

导出成 PDF”，系统会选择 XLSX Skill，并调用对应的导出脚本生成 PDF 文件。生成完成后，系统检查输出文件是否存在，并将结果返回用户。

如果用户请求无法通过固定 Action 直接完成，则系统进入 Agent Loop。LLM 根据当前上下文和工具定义决定是否调用工具。每次工具调用后，工具执行结果都会返回给 LLM，LLM 再根据结果继续推理，直到生成最终回复。

整个交互流程体现了 Agent 的基本特征：感知用户输入、理解任务意图、选择工具执行、观察工具结果、生成最终回复。与传统 QQBot 相比，该系统不再是简单的命令响应，而是具有任务理解和执行闭环的智能体。



3.3 数据流设计

系统的数据流主要包括输入数据、中间数据、执行数据和输出数据。

输入数据来自 QQ 用户，包括文本消息、上传文件、用户 ID、会话 ID 和历史对话。文本消息用于表达用户意图，上传文件用于文件问答或文件处理任务，用户 ID 和会话 ID 用于区分不同用户和不同会话。

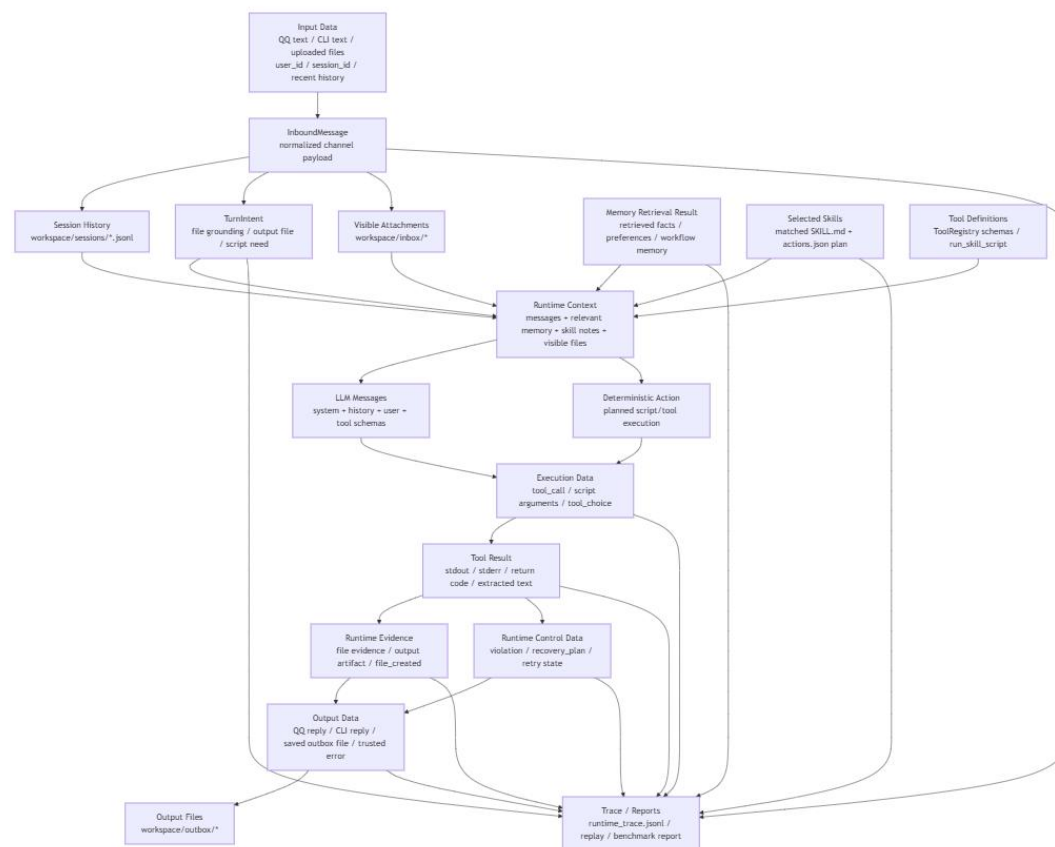
中间数据主要包括 InboundMessage、TurnIntent、Session History、Memory Retrieval Result、Selected Skills、Tool Definitions 和 LLM Messages。

InboundMessage 是统一消息格式，TurnIntent 是任务理解结果，Session History 和 Memory Retrieval Result 为 Agent 提供上下文，Selected Skills 和 Tool Definitions 则决定 Agent 可使用的能力范围。

执行数据包括 Tool Call、Tool Result、Script Return Code、File Evidence、Output Artifact、Runtime Violation 和 Recovery Plan。这些数据体现了 Agent 是否真实执行了任务。例如，文件总结任务中的 File Evidence 表示系统读取到的文件内容；文件转换任务中的 Output Artifact 表示生成的输出文件。

输出数据包括 QQ 文本回复、生成文件、错误提示、任务完成摘要和 Trace 日志。对于普通问答任务，系统输出文本回复；对于文件处理任务，系统输出生成文件路径或文件本身；对于失败任务，系统输出明确错误原因；对于开发者，系统还会保留 Trace 和 Benchmark 报告。

Data Flow



四、关键实现与代码展示

4.1 QQBot 消息接入实现

QQBot 消息接入是系统运行的第一步。该部分负责监听 QQ 用户消息，并提取用户文本、用户 ID、会话 ID 和附件信息。为了保证后续 Agent 逻辑不依赖具体平台，QQBot Channel 不直接执行业务逻辑，而是将原始 QQ 消息转换为统一的 InboundMessage。

这种设计将平台接入与 Agent 核心逻辑分离。一方面，QQBot Channel 只需要关注 QQ 平台消息格式、文件下载和消息回复；另一方面，MiniAgentApp 只需要处理统一的 InboundMessage，而不需要关心消息来自 QQ、CLI 还是其他平台。未来如果需要支持微信、飞书或 Web UI，也只需要新增对应 Channel Adapter，而不需要重写 Agent 核心逻辑。

[message.py]

```
@dataclass

class InboundMessage:

    """从外部收到的消息"""

    channel: str

    sender_id: str

    chat_id: str

    content: str

    attachments: list[Attachment] = field(default_factory=list)

    metadata: dict[str, Any] = field(default_factory=dict)
```

```
@property

def session_key(self) -> str:

    return f"{self.channel}:{self.chat_id}"
```

```
@property
```

```

    def media(self) -> list[str]:

        return [item.path for item in self.attachments]

@dataclass
class OutboundMessage:

    """要发送到外部的消息"""

    channel: str

    chat_id: str

    content: str

    attachments: list[Attachment] = field(default_factory=list)

    metadata: dict[str, Any] = field(default_factory=dict)

```

```

@property

def media(self) -> list[str]:

    return [item.path for item in self.attachments]

```

```

class MessageBus:

    """消息总线, 负责在不同渠道之间传递消息"""

```

```

def __init__(self):

    self.inbound: asyncio.Queue[InboundMessage] = asyncio.Queue()

    self.outbound: asyncio.Queue[OutboundMessage] = asyncio.Queue()

```

```

async def publish_inbound(self, msg: InboundMessage):

    await self.inbound.put(msg)

```

```

async def consume_inbound(self) -> InboundMessage:

    return await self.inbound.get()

```

```
async def publish_outbound(self, msg: OutboundMessage):  
  
    await self.outbound.put(msg)
```

```
async def consume_outbound(self) -> OutboundMessage:  
  
    return await self.outbound.get()
```

4.2 工具定义

ToolRegistry 是系统统一管理工具的模块。所有外部能力都通过 ToolRegistry 注册和调用，包括文件读取、文件写入、文件查找、代码搜索、脚本执行、上传文件读取和输出文件保存等。

这种设计有两个优点。第一，它为 LLM 提供了统一的工具接口，使 LLM 不需要直接操作底层文件系统。第二，它便于系统记录和检查工具调用行为。每次工具调用都可以被记录为 Tool Call，每次工具执行结果都可以被记录为 Tool Result，从而支持 RuntimeVerifier 和 Trace 分析。

定义基类: [base.py (line 7)]

```
class Tool(ABC):  
  
    """工具基类, 所有工具都需要继承这个类"""  
  
    @property  
    @abstractmethod  
    def name(self) -> str: ...
```

```
    @property  
    @abstractmethod  
    def description(self) -> str: ...
```

```
@property

@abstractmethod

def parameters(self) -> dict[str, Any]: ...
```

```
@abstractmethod

async def execute(self, **kwargs) -> str: ...
```

```
def to_schema(self) -> dict[str, Any]:

    return {

        "type": "function",

        "function": {

            "name": self.name,

            "description": self.description,

            "parameters": self.parameters,

        },

    }
```

注册表实现: [registry.py (line 8)]

```
class ToolRegistry:

    """工具注册表, 管理可用的工具"""

    def __init__(self):

        self._tools: dict[str, Tool] = {}
```

```
def register(self, tool: Tool):

    self._tools[tool.name] = tool
```

```
def get_definitions(self) -> list[dict[str, Any]]:
```

```
return [tool.to_schema() for tool in self._tools.values()]
```

```
async def execute(self, name: str, params: dict[str, Any]) -> str:

    tool = self._tools.get(name)

    if not tool:

        return f"Error: Tool '{name}' not found."

    if not isinstance(params, dict):

        return (

            "Error: invalid tool arguments: expected a JSON object "

            f"but got {type(params).__name__}."

        )

    try:

        return await tool.execute(**params)

    except Exception as exc:

        return f"Error: {exc}"
```

4.3 TurnIntent 意图识别实现

TurnIntent 是本项目中连接用户自然语言和 Agent 执行流程的重要模块。用户的自然语言请求往往是不结构化的，而 Agent 执行任务需要明确知道当前任务类型。因此，TurnIntent 模块将用户请求转化为结构化任务信息。

例如，当用户输入“帮我总结这个 PDF”时，系统需要判断该任务属于文件问答任务，并设置 `requires_file_grounding` 为 `true`。当用户输入“把这个 Excel 导出成 PDF”时，系统需要判断该任务属于文件转换任务，并设置 `requires_output_file` 和 `requires_script` 为 `true`，同时识别源文件格式为 `xlsx`，目标格式为 `pdf`。

TurnIntent 的作用不是生成最终回答，而是为后续执行流程提供控制信号。它决定系统是否需要读取文件、是否需要调用脚本、是否需要生成输出文件以及应该选择哪个 Skill。因此，TurnIntent 是 QQBot Agent 从自然语言理解走向真实任务执行的关键环节。

[intent.py (line 19)]

```
@dataclass

class TurnIntent:

    """Structured runtime intent used by gates instead of scattered prompt hints."""

    operation: str = "answer"

    requires_file_grounding: bool = False

    requires_output_file: bool = False

    requires_script: bool = False

    target_format: str = ""

    source_format: str = ""

    is_outbox_listing: bool = False

    confidence: float = 0.5

    reasons: list[str] = field(default_factory=list)
```

```
def to_trace_dict(self) -> dict[str, object]:

    return asdict(self)
```

```
def infer_turn_intent(

    user_text: str,

    *,

    attachments: Iterable[Attachment] | None = None,

    has_visible_attachments: bool = False,

    script_skill_names: Iterable[str] | None = None,

) -> TurnIntent:

    text = _normalize(_extract_user_request(user_text))

    attachment_list = list(attachments or [])

    has_files = bool(has_visible_attachments or attachment_list)
```

```
script_skills = {str(name).lower() for name in script_skill_names or []}

target_format = _target_format(text)

source_format = _source_format(text, attachment_list, target_format)

intent = TurnIntent(source_format=source_format, target_format=target_format)
```

4.4 Skill 配置与执行实现

支持 SKILL.md + actions.json + scripts/ 的能力包结构, 通过 Skill Router 自动选择, 并统一通过 run_skill_script 执行脚本。Skill 是本项目实现任务扩展的重要机制。每个 Skill 面向一类具体任务, 例如 PDF Skill 负责 PDF 读取、合并、抽页和旋转, XLSX Skill 负责 Excel 读取、修改和导出 PDF, DOCX Skill 负责 Word 文档读取、生成和修订处理, Code Navigation Skill 负责代码搜索和项目结构分析。

每个 Skill 通常由 SKILL.md、actions.json 和 scripts 目录组成。SKILL.md 用于描述该 Skill 的能力和方式使用, actions.json 用于定义可自动执行的动作, scripts 目录中存放具体执行脚本。通过这种组织方式, 系统可以方便地新增或修改 Skill, 而不需要重构 Agent 主流程。

当用户提出明确任务时, 例如“合并这两个 PDF”, 系统可以通过 TurnIntent 判断任务类型, 再由 SkillRouter 选择 PDF Skill, 随后 ActionPlanner 根据 actions.json 匹配 merge_pdfs 动作, 并调用对应脚本完成任务。脚本执行完成后, 生成文件被保存到 outbox, 并由 QQBot 返回给用户。

Skill 装载与注入

```
class SkillLoader:

    """Facade for the project skill runtime."""

    def __init__(

        self,
```

```

        workspace: Path,

        builtin_dir: Path | None = None,

        *,

        runtime_workspace: Path | None = None,

        trace_sink: Any | None = None,

        route_mode: str = SKILL_ROUTE_MODE,

        llm_client: Any = client,

        model: str = MODEL,

    ):

        self.workspace = workspace

        self.runtime_workspace = runtime_workspace or workspace

        self.trace_sink = trace_sink

        self.workspace_skills = workspace / "skills"

        self.builtin_skills = builtin_dir

        self.scanner = SkillScanner(self.workspace_skills, self.builtin_skills)

        self.registry = SkillRegistry(self.scanner)

        self.router = SkillRouter(mode=route_mode, llm_client=llm_client, model=model)

        self.policy = SkillPolicyEngine()

        self.runtime = SkillRuntime(self.registry, self.runtime_workspace)

        self.trace = SkillTraceLogger(self.runtime_workspace)

```

对 LLM 暴露的统一工具

```

class RunSkillScriptTool(Tool):

    """统一执行 skill 目录内脚本的工具。"""

    def __init__(self, skill_loader: SkillLoader):

        self.skill_loader = skill_loader

```

```
@property

def name(self) -> str:

    return "run_skill_script"
```

```
@property

def description(self) -> str:

    return (

        "Run a Python script from a selected skill's scripts/ directory with path validation,"
        "
        "execution tracing, and captured stdout/stderr. Use the normal tool parameter fields:"
        "
        "skill_name, script_path, arguments, timeout_seconds, and optionally cwd."

    )
```

```
@property

def parameters(self) -> dict[str, Any]:

    ...
```

```
async def execute(

    self,

    skill_name: str,

    script_path: str,

    arguments: list[str] | None = None,

    timeout_seconds: int = 60,

    cwd: str | None = None,

    **kwargs,

) -> str:

    try:

        return await self.skill_loader.run_skill_script(
```

```
        skill_name=skill_name,\n        script_path=script_path,\n        arguments=arguments or [],\n        timeout_seconds=timeout_seconds,\n        cwd=cwd,\n    )\n\n    except Exception as exc:\n\n        return f"Error: {exc}"
```

4.5 MiniAgentApp 主流程实现

MiniAgentApp 是本项目的核心控制模块。它负责处理一次完整的用户请求，包括保存附件、读取会话历史、判断任务意图、选择 Skill、构造 LLM 上下文、调用工具循环、执行结果检查和最终回复生成。

当 MiniAgentApp 收到 InboundMessage 后，首先会处理用户上传的附件，将文件保存到 workspace 对应目录，并记录到当前会话中。然后，系统读取当前 session 的历史消息，使 Agent 能够理解多轮上下文。接着，MiniAgentApp 调用 TurnIntent 模块判断当前任务是否需要文件证据、是否需要输出文件、是否需要脚本执行。

如果任务需要文件证据，MiniAgentApp 会先读取用户上传文件内容，并将其加入上下文。如果任务需要调用 Skill，系统会通过 SkillRouter 查找相关 Skill，并在必要时调用 ActionPlanner 生成执行计划。对于无法直接执行的任务，MiniAgentApp 会进入 Agent Loop，由 LLM 根据工具定义进行多步工具调用。

最终，MiniAgentApp 会调用 RuntimeVerifier 检查任务是否完成。如果通过检查，则生成最终回复；如果未通过，则调用 RuntimeRecovery 进行修复或返回明确失败说明。整个流程会写入 Trace，供后续调试和评估使用。

[app.py (line 865)]

```
class MiniAgentApp:\n\n    """把消息总线、记忆、工具和 Channel 装配成可运行的 Agent 应用。"""
```

```

def __init__(
    self,

    workspace: Path = WORKSPACE,

    model: str = MODEL,

    llm_client: Any = client,

    tools: ToolRegistry | None = None,

    bus: MessageBus | None = None,

    skill_loader: SkillLoader | None = None,

    context_builder: ContextBuilder | None = None,

    attachment_store: AttachmentStore | None = None,

    memory: MemoryStore | None = None,

    sessions: SessionManager | None = None,

    trace_sink: Any | None = None,
):
    self.workspace = workspace

    self.model = model

    self.client = llm_client

    self.tools = tools

    self.bus = bus or MessageBus()

    self.skill_loader = skill_loader or SkillLoader(workspace)

    self.ctx = context_builder or ContextBuilder(workspace, skill_loader=self.skill_loader)

    self.attachment_store = attachment_store or AttachmentStore(workspace)

    self.memory = memory or MemoryStore(workspace)

    self.sessions = sessions or SessionManager(workspace)

    self.trace_sink = trace_sink

    self.channels: dict[str, BaseChannel] = {}

```

```

def register_channel(self, channel: BaseChannel):

```

```
async def handle_inbound(self, inbound: InboundMessage):

async def inbound_worker(self):

async def run(self, channels: list[BaseChannel]):
```

4.6 Agent 核心循环

Agent 核心循环负责大语言模型与工具之间的交互。系统首先将用户请求、会话历史、文件上下文、可用工具和相关 Skill 信息组织成 LLM 输入。LLM 根据当前上下文判断是否需要调用工具。如果 LLM 请求工具调用，系统会通过 ToolRegistry 执行对应工具，并将工具执行结果返回给 LLM。LLM 再根据工具结果继续推理，直到生成最终回答。

该循环使 QQBot Agent 具备了任务执行能力。与普通聊天机器人不同，Agent 不只是一次性生成回答，而是可以在回答前主动调用文件读取工具、代码搜索工具或 Skill 脚本。例如，在文件总结任务中，Agent 可以先调用文件读取工具获取文档内容，再基于内容生成总结；在代码搜索任务中，Agent 可以先调用 search_code 工具找到相关代码位置，再向用户解释代码逻辑。

Agent 核心循环中的每次 LLM 请求、工具调用和工具返回都会被记录到 Trace 中。这样不仅有助于调试，也能为后续 Benchmark 和 Replay 提供依据。

[app.py (line 101)]

```
async def agent_loop(

    client: Any,

    model: str,

    messages: list[dict[str, Any]],

    tools: ToolRegistry,

    max_iterations: int = 10,

    metrics: dict[str, Any] | None = None,

    trace_sink: Any | None = None,

    requires_file_grounding: bool = False,

    initial_file_grounding_evidence: bool = False,
```

```

requires_output_file: bool = False,

force_skill_script_tool: bool = False,

) -> str:

    recovery = RuntimeRecoveryController(max_retries=2)

    tools_executed_in_turn = 0

    tool_names_executed_in_turn: list[str] = []

    file_grounding_evidence_collected = initial_file_grounding_evidence

    output_file_created = False

    skill_script_succeeded = False

    for iteration_index in range(max_iterations):

        iteration = iteration_index + 1

        if tool_calls:

            ...

            result = await tools.execute(tc.function.name, params or {})

            ...

            messages.append({"role": "tool", "tool_call_id": tc.id, "content": result})

            continue

        reply = msg.content or ""

        verification = verify_final_reply(

            latest_user=_latest_user_text(messages),

            reply=reply,

            state=RuntimeVerificationState(...),

            grounding_detector=_detect_grounding_violation,

        )

        if not verification.ok:

            plan = recovery.plan(verification)

            if plan.should_fail:

```

```

        return plan.error_message

    if plan.should_retry:

        recovery.apply_plan(plan)

        messages.append({"role": "assistant", "content": reply})

        messages.append({"role": "system", "content": plan.system_message})

        continue

    return reply

return "Max iterations reached."

```

4.7 RuntimeVerifier 与 RuntimeRecovery 实现

RuntimeVerifier 是 QQBot Agent 内部的执行检查模块。由于本项目中的 Agent 需要处理真实文件和生成真实结果，因此系统不能只依赖模型的自然语言声明，而需要检查任务执行结果是否与用户请求一致。

对于文件问答任务，RuntimeVerifier 会检查本轮任务是否读取了文件内容。如果用户要求总结 PDF，但系统没有任何文件读取记录，则说明该任务不应直接返回总结。对于文件生成任务，RuntimeVerifier 会检查是否存在真实输出产物。如果用户要求导出 PDF，但 outbox 中没有生成文件，则系统不能回复“已完成”。对于脚本型任务，RuntimeVerifier 还会检查是否调用了对应 Skill 脚本。

RuntimeRecovery 则用于处理验证失败或工具失败的情况。当系统发现缺少文件证据时，Recovery 会引导 Agent 先读取文件；当系统发现缺少输出文件时，Recovery 会引导 Agent 调用脚本生成文件；当工具执行失败时，Recovery 会将错误信息反馈给 Agent，使其调整参数或返回明确失败说明。

这两个模块的作用是支撑 QQBot Agent 更稳定地完成真实任务，而不是作为独立系统存在。

[runtime_verifier.py (line 19)]

```

@dataclass
class RuntimeVerificationState:

```

```

requires_file_grounding: bool = False

file_grounding_evidence_collected: bool = False

requires_output_file: bool = False

output_file_created: bool = False

force_skill_script_tool: bool = False

skill_script_succeeded: bool = False

tools_executed_in_turn: int = 0

tool_names_executed_in_turn: list[str] = field(default_factory=list)

def verify_final_reply(
    *,
    latest_user: str,
    reply: str,
    state: RuntimeVerificationState,
    grounding_detector: GroundingDetector | None = None,
) -> VerificationResult:
    tool_names = list(state.tool_names_executed_in_turn)

    if state.force_skill_script_tool and not state.skill_script_succeeded:
        return VerificationResult.failed(
            violation_type="script_tool_required_without_success",
            trace_kind="script_tool_violation",
            finish_reason="script_tool_violation",
            recovery_kind="force_skill_script",
            ...
        )

    if state.requires_file_grounding and not state.file_grounding_evidence_collected:
        return VerificationResult.failed(
            violation_type="file_grounding_required_without_evidence",

```

```

        trace_kind="grounding_violation",

        finish_reason="grounding_violation",

        recovery_kind="force_file_grounding",

        ...

    )

    if state.requires_output_file and not state.output_file_created:

        return VerificationResult.failed(

            violation_type="output_file_required_without_artifact",

            trace_kind="output_violation",

            finish_reason="output_violation",

            recovery_kind="force_output_file",

            ...

        )

    if looks_like_output_file_claim(reply) and not state.output_file_created:

        return VerificationResult.failed(

            violation_type="output_file_claim_without_artifact",

            trace_kind="output_violation",

            finish_reason="output_violation",

            recovery_kind="correct_output_claim",

            ...

        )

    return VerificationResult.passed()

```

[runtime_recovery.py (line 40)]

```

def apply_plan(self, plan: RecoveryPlan) -> None:

    if plan.should_retry:

```

```
self.state.next_forced_tools = list(plan.forced_tools or [])

def consume_forced_tools(self) -> list[str]:

    tools = list(self.state.next_forced_tools or [])

    self.state.next_forced_tools = None

    return tools
```

4.8 Harness 系统实现

[harness/config.py (line 11)]

```
@dataclass
class HarnessConfig:

    workspace: Path = WORKSPACE

    model: str = MODEL

    llm_client: Any = client

    results_dir: Path = WORKSPACE / "benchmarks" / "results"

    tmp_dir: Path = WORKSPACE / "benchmarks" / "tmp"

    isolated: bool = False
```

[runtime.py (line 21)]

```
@dataclass
class RuntimeComponents:

    bus: MessageBus

    attachment_store: AttachmentStore

    skill_loader: SkillLoader

    context_builder: ContextBuilder

    memory: MemoryStore

    sessions: SessionManager

    trace_sink: TraceSink
```

```

class MiniAgentHarness:

    def __init__(self, config: HarnessConfig | None = None):

        self.config = config or HarnessConfig()

        self.assembly = AgentAssembly(self.config)

    def build_app(self) -> MiniAgentApp:

        context = RuntimeContext.live(...)

        components = self.assembly.build_components(context)

        return self.assembly.build_app(context=context, components=components)

    def build_eval_session(...):

        context = RuntimeContext.eval_task(...)

        return self.build_runtime_session(context)

    async def run_live(self, channel_names: list[str]) -> None:

        app = self.build_app()

        channels = self.assembly.build_channels(app, channel_names)

        await app.run(channels)

    async def run_eval_turn(...):

        session = self.build_eval_session(...)

        return await session.run_turn(...)

```

4.9 配置文件

[config.py (line 1)]

```

API_KEY = os.environ.get("DASHSCOPE_API_KEY")

BASE_URL = os.environ.get(
    "DASHSCOPE_BASE_URL",
    "https://dashscope.aliyuncs.com/compatible-mode/v1",
)

MODEL = os.environ.get("DASHSCOPE_MODEL", "qwen-plus-2025-09-11")

EMBEDDING_MODEL = os.environ.get("DASHSCOPE_EMBEDDING_MODEL", "text-embedding-v4")

PROJECT_ROOT = Path(__file__).resolve().parents[2]

BASE_DIR = PROJECT_ROOT

WORKSPACE = PROJECT_ROOT / "workspace"

PREFERRED_PYTHON = "3.11.15"

REQUIRED_PYTHON_SERIES = (3, 11)

PREFERRED_PYTHON_EXECUTABLE = Path(r"D:\conda\envs\assistant\python.exe")

MAX_HISTORY_MESSAGES = 15

MEMORY_CONSOLIDATE_TRIGGER = 30

MEMORY_KEEP_RECENT = 15

MEMORY_RETRIEVAL_TOP_K = 4

MEMORY_RETRIEVAL_CANDIDATES = 8

SKILL_ROUTE_MODE = "hybrid" # 可选: "hybrid" / "rule" / "llm"

CHANNELS: dict[str, dict[str, Any]] = {
    "qq": {
        "enabled": os.environ.get("QQ_BOT_ENABLED", "true").lower() in {"1", "true", "yes", "on"},
        "appId": os.environ.get("QQ_BOT_APP_ID", ""),
        "secret": os.environ.get("QQ_BOT_SECRET", ""),
        "allowFrom": [],
        "msgFormat": "plain",
        "ackMessage": os.environ.get("QQ_BOT_ACK_MESSAGE", "🔄 Processing..."),
    }
}

```

```
}

QQ_CHANNEL = CHANNELS["qq"]

def ensure_supported_python() -> None:

    current_series = sys.version_info[:2]

    if current_series != REQUIRED_PYTHON_SERIES:

        raise RuntimeError(

            "MiniAgent requires Python "

            f"{REQUIRED_PYTHON_SERIES[0]}.{REQUIRED_PYTHON_SERIES[1]} "

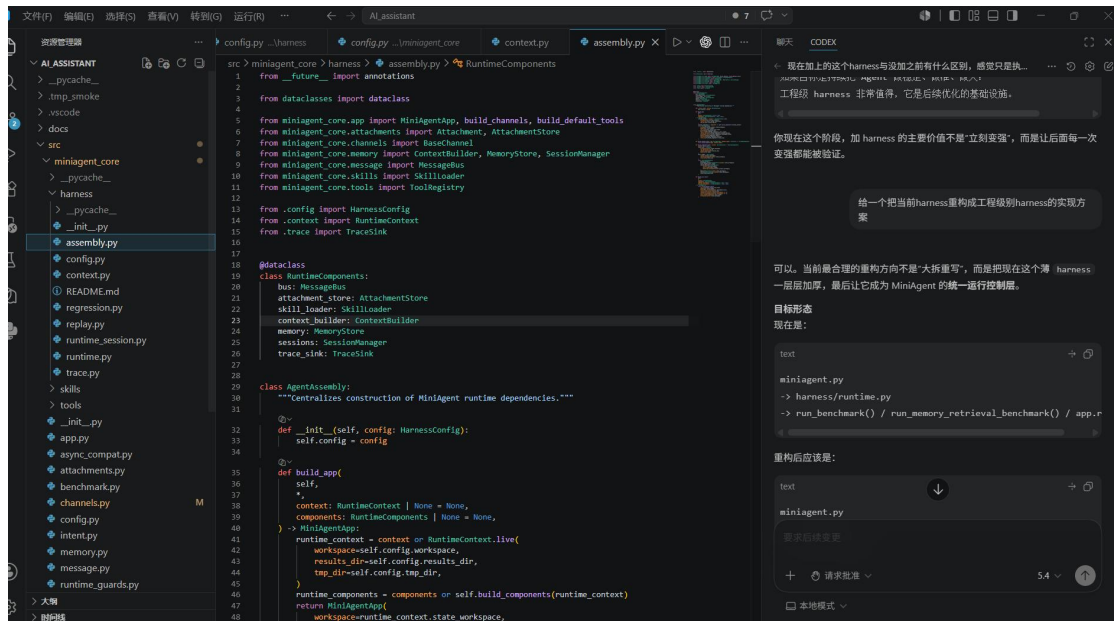
            f"(preferred environment: {PREFERRED_PYTHON} at {PREFERRED_PYTHON_EXECUTABLE}). "

            f"Current interpreter: {sys.executable} ({sys.version.split()[0]})."

        )

client: Any = OpenAI(api_key=API_KEY, base_url=BASE_URL) if OpenAI else None
```

4.4 AI IDE 使用截图



聊天CODEX

← 现在加上的这个harness与没加之前有什么区别，感觉只是执行方式变了

```
powershell

D:\conda\envs\assistant\python.exe -m compileall miniagent.py miniagent_core
D:\conda\envs\assistant\python.exe miniagent.py harness --help
D:\conda\envs\assistant\python.exe miniagent.py harness eval --limit 0
D:\conda\envs\assistant\python.exe miniagent.py benchmark --limit 0
```

都通过。python 默认指向系统 Python 3.7，会被项目环境检查拦住，这是原有约束，不是这次改动导致的。

有个小提醒：跑 harness eval --limit 0 触发了现有 benchmark 的默认任务初始化，所以 workspace/benchmarks/tasks.json 和 workspace/benchmarks/fixtures/ 现在显示为未跟踪文件。核心重构文件都在 miniagent_core/harness/、miniagent_core/benchmark.py 和 miniagent.py。下一步可以继续做 replay.py / regression.py，或者先把 --isolated 真正接到 CLI 和 eval workspace 隔离上。

已编辑 9 个文件
+454 -161

撤销 审核

miniagent_core/harness/config.py+17 -0

miniagent_core/harness/context.py+69 -0

miniagent_core/harness/trace.py+60 -0

再显示 6 个文件

file_grounding_required_without_evidence需要文件证据但没有读文件

output_file_required_without_artifact需要输出文件但没有真实产物

output_file_claim_without_artifact模型声称已保存文件但没有产物

file_content_claim_without_evidence模型声称看过文件但没有证据

tool_claim_without_tool_execution模型声称调用工具但工具调用数为 0

代码中 RuntimeVerificationState 保存了当前轮的证据状态，例如是否需要文件证据、是否已经收集文件证据、是否需要输出文件、是否已经生成文件、脚本是否成功、执行过哪些工具等。verify_final_reply() 会基于这些状态判断是否放行最终回复。[GitHub](#)

这解决了 Agent 项目里非常常见的问题：
模型说“我已读取文件 / 已保存文件 / 已执行脚本”，但实际上什么都没做。

3.5 runtime_recovery.py：失败恢复和强制重试

RuntimeRecovery 负责 verifier 发现问题后的处理策略。

例如，如果本轮需要 skill script，但模型没有成功调用 run_skill_script，recovery 会追加 system message，明确要求模型必须调用 run_skill_script，并且只有在返回 Return code: 0 后才能最终回答。超过最大重试次数后，它不会放行模型回复，而是返回明确错误。[GitHub](#)

对文件 grounding 也是类似：如果需要文件证据但没读文件，它会要求模型调用 list_uploaded_files 和 read_uploaded_file，或者用 run_skill_script 做 PDF 精确提取。[GitHub](#)

这部分可以理解为 Agent 自我修复控制器，但它不是让模型自己反思，而是由 runtime 根据违规类型执行确定性恢复策略。

4. Skill 系统分析

Skill 系统把通用 Agent 扩展成项目级任务 Agent，文档中说明，它会扫描 workspace/skills/*，路由由用户请求，按需注入 SKILL.md，用 actions.json 生成确定性 action，并统一通过 run_skill_script 执行脚本。[GitHub](#)

当前 workspace skills 包括：

↓

+ 有问题，尽管问

均衡 语音 静音

ChatGPT 也可能犯错。请检查重要信息。

五、测试与评估

5.1 功能测试

为了验证 QQBot Agent 的功能完整性，本项目可以从文本问答、文件上传、文件总结、文件转换、PDF 处理、代码搜索和失败恢复等方面进行测试。

首先是 QQ 文本问答测试。该测试用于验证 QQBot 是否能够正常接收用户文本消息，并通过 Agent 生成合理回复。用户可以发送“你能做什么”之类的问题，系统应能够说明自身支持文件处理、代码搜索、天气查询等能力。

其次是文件上传与识别测试。用户通过 QQ 上传 PDF、Word 或 Excel 文件后，系统应能够保存文件，并将文件信息与当前会话关联。后续当用户使用“这个文件”“刚才的表格”等表达时，Agent 应能够找到对应文件。

第三是 PDF 文件总结测试。用户上传 PDF 后发送“帮我总结这个 PDF”。系统应判断这是文件问答任务，读取 PDF 内容，并基于文件内容生成总结。该测试的重点不是最终回答是否流畅，而是 Agent 是否真正读取了文件，并基于文件内容回答。

第四是 Excel 导出 PDF 测试。用户上传 Excel 文件后发送“把这个 Excel 导出成 PDF”。系统应判断这是文件转换任务，选择 XLSX Skill，调用导出 PDF 脚本，并在 outbox 中生成 PDF 文件。测试时需要检查工具调用记录、脚本返回结果和输出文件是否真实存在。

第五是 PDF 合并测试。用户上传两个 PDF 文件后要求合并。系统应识别多个 PDF 输入，选择 PDF Skill，调用合并脚本，并生成合并后的 PDF 文件。该测试可以验证 Agent 对多文件任务的处理能力。

第六是代码搜索测试。用户提出“帮我找一下 agent_loop 是在哪里实现的”之类的问题。Agent 应调用代码搜索工具或文件查找工具，返回真实代码路径和相关说明，而不是凭空猜测。

第七是失败恢复测试。用户上传不支持的文件格式，或者提出当前系统无法完成的任务。系统应检测失败原因，避免回复“已完成”，并通过 RuntimeRecovery 返回明确的失败说明或修复建议。

```
[benchmark.py]
```

```
@dataclass
```

```

class BenchmarkTask:

    id: str

    category: str

    prompt: str

    attachments: list[dict[str, str]] = field(default_factory=list)

    expected_reply_contains: list[str] = field(default_factory=list)

    expected_reply_contains_any: list[str] = field(default_factory=list)

    expected_tools_all: list[str] = field(default_factory=list)

    expected_tools_any: list[str] = field(default_factory=list)

    expected_outbox_suffixes: list[str] = field(default_factory=list)

    expected_outbox_files: list[dict[str, Any]] = field(default_factory=list)

    expected_max_tool_calls: int | None = None

    max_iterations: int = 10

```

```

async def run_benchmark(

    *,

    tasks_file: Path = DEFAULT_TASKS_FILE,

    results_dir: Path = DEFAULT_RESULTS_DIR,

    limit: int | None = None,

    delay_seconds: float = 3.0,

    model: str = MODEL,

    harness: Any | None = None,

) -> dict[str, Any]:

    tasks = load_tasks(tasks_file)

    ...

    for index, task in enumerate(tasks, start=1):

        if index > 1 and delay_seconds > 0:

            await asyncio.sleep(delay_seconds)

        result = await run_task(task, run_id=run_id, model=model, harness=harness)

```

```
task_results.append(result)

summary = build_summary(task_results, run_id=run_id, tasks_file=tasks_file)

...
```

5.2 Agent 行为评估

Agent 行为评估不能只看最终自然语言回复，还需要关注整个执行过程。对于本项目，可以从意图识别准确性、工具调用正确性、文件证据正确性、输出产物正确性、回复质量、失败恢复能力和 Trace 完整性几个方面进行评估。

意图识别准确性用于衡量 TurnIntent 是否正确判断用户任务。例如，文件总结任务是否被识别为文件问答，Excel 导出 PDF 是否被识别为文件转换，代码定位任务是否被识别为代码搜索。工具调用正确性用于衡量 Agent 是否选择了正确工具或 Skill，例如 PDF 任务是否调用 PDF Skill，Excel 任务是否调用 XLSX Skill，代码搜索任务是否调用 search_code。

文件证据正确性用于评估文件问答任务中系统是否真实读取了文件。如果用户要求总结 PDF，但 Trace 中没有文件读取记录，则该任务不能被视为成功。输出产物正确性用于评估文件生成任务是否真的生成了文件，例如 outbox 中是否存在生成的 PDF，文件是否能够打开，内容是否符合任务要求。

回复质量用于评估 Agent 最终返回给 QQ 用户的信息是否清晰、准确、完整。对于成功任务，回复应说明任务完成情况和输出文件位置；对于失败任务，回复应说明失败原因，而不是模糊地表示任务已完成。失败恢复能力用于评估工具失败后系统是否能够重新尝试或给出可信失败说明。Trace 完整性则用于评估运行过程是否记录了关键事件，是否便于后续调试和复现。

```
def evaluate_task(

    task: BenchmarkTask,

    reply: str,

    events: list[ToolEvent],

    outbox_files: list[str],

    metrics: dict[str, Any],
```

```

) -> list[str]:

    failure_types: list[str] = []

    reply_text = str(reply or "")

    reply_lower = reply_text.lower()

    tool_names = [event.tool for event in events]

    if client is None:

        failure_types.append("llm_unavailable")

    if reply_text.startswith("Error:"):

        failure_types.append("final_error")

    if "Max iterations reached" in reply_text:

        failure_types.append("max_iterations")

    if str(metrics.get("finish_reason", "")) in {"forced_tool_use_failed",
"incomplete_progress", "max_iterations"}:

        failure_types.append(str(metrics["finish_reason"]))

```

```

for expected in task.expected_reply_contains:

    if str(expected).lower() not in reply_lower:

        failure_types.append("missing_expected_reply")

        break

```

```

if task.expected_reply_contains_any and not any(

    str(expected).lower() in reply_lower for expected in
task.expected_reply_contains_any

):

    failure_types.append("missing_expected_reply")

```

```

for expected_tool in task.expected_tools_all:

    if expected_tool not in tool_names:

```

```
failure_types.append("missing_expected_tool")
```

```
break
```

```
if task.expected_tools_any and not any(tool in tool_names for tool in
task.expected_tools_any):
```

```
failure_types.append("missing_expected_tool")
```

```
if task.expected_max_tool_calls is not None and len(events) >
task.expected_max_tool_calls:
```

```
failure_types.append("too_many_tool_calls")
```

```
for suffix in task.expected_outbox_suffixes:
```

```
normalized_suffix = str(suffix).lower()
```

```
if not any(Path(path).suffix.lower() == normalized_suffix for path in
outbox_files):
```

```
failure_types.append("missing_expected_outbox")
```

```
break
```

```
for expectation in task.expected_outbox_files:
```

```
matched, file_failures = evaluate_outbox_file_expectation(expectation,
outbox_files)
```

```
if not matched:
```

```
failure_types.extend(file_failures or ["missing_expected_outbox_file"])
```

```
blocking_tool_failures = [
```

```
event.failure_type
```

```
for event in events
```

```
if event.failed and event.failure_type in {"tool_not_found",
"invalid_tool_arguments", "skill_script_not_found"}
```

```
]
```

```

if blocking_tool_failures:
    failure_types.extend(blocking_tool_failures)

metric_tool_failures = []

for item in metrics.get("tool_errors", []) or []:
    if not isinstance(item, dict):
        continue

    _, failure_type = classify_tool_failure(str(item.get("tool") or ""),
str(item.get("preview") or ""))

    if failure_type in {"tool_not_found", "invalid_tool_arguments",
"skill_script_not_found"}:

        metric_tool_failures.append(failure_type)

if metric_tool_failures:
    failure_types.extend(metric_tool_failures)

return dedupe(failure_types)

```

5.3 Benchmark

为了系统化评估 QQBot Agent，可以设计一个包含多种任务类型的 Benchmark。每个测试样例记录任务编号、任务类型、用户输入、输入文件、期望工具、实际工具、是否需要文件证据、是否需要输出文件、是否通过和失败原因。

Benchmark 可以覆盖文件总结、Excel 导出 PDF、PDF 合并、代码搜索、天气查询和失败恢复等任务。对于每个任务，不仅要检查最终回复，还要检查 Trace 中是否出现期望工具调用，以及输出文件是否真实生成。

例如，在“总结 PDF”任务中，通过标准应包括：TurnIntent 识别为文件问答任务，系统读取 PDF 文件，最终回复包含基于文件内容的总结。在“Excel 导出 PDF”任务中，通过标准应包括：TurnIntent 识别为文件转换任务，系统选择 XLSX Skill，调用导出脚本，outbox 中生成 PDF 文件，最终回复给出生成结果。在“代码搜

索”任务中，通过标准应包括：系统调用代码搜索工具，返回真实文件路径和相关代码说明。

通过 Benchmark，可以更加客观地评价 Agent 的实际能力，而不是只依赖单次 Demo 展示。

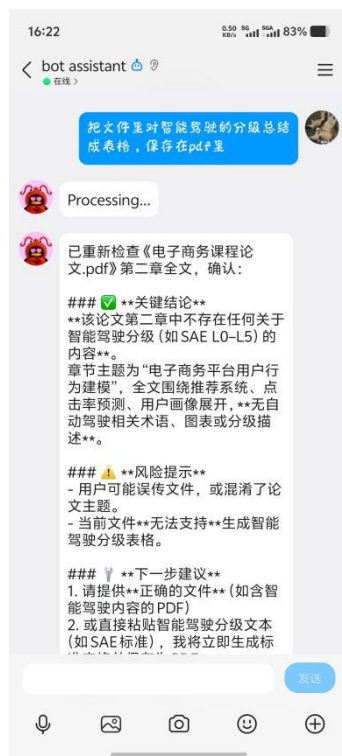
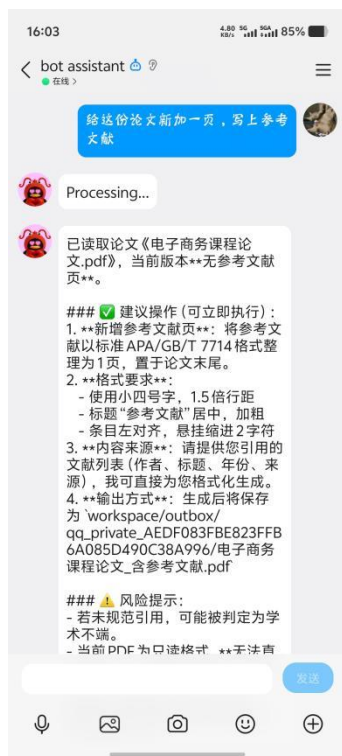
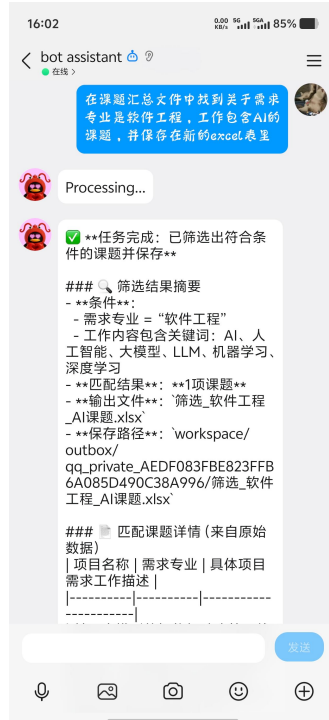
完整代码见 [cs599-project/src/miniagent_core/benchmark.py](https://github.com/dwjds/cs599-project/blob/main/src/miniagent_core/benchmark.py) at [main · dwjds/cs599-project](https://github.com/dwjds/cs599-project)

```
workspace > benchmarks > results > latest.md > abc # MiniAgent Benchmark Report
1  # MiniAgent Benchmark Report
2
3  - Run ID: 20260510_194443
4  - Tasks: 20
5  - Passed: 16
6  - Failed: 4
7  - Success Rate: 80.00%
8  - Total Tool Calls: 27
9  - Avg Tool Calls: 1.35
10 - Avg Steps: 2.4
11
12 ## Failure Types
13
14 - missing_expected_reply: 1
15 - missing_expected_tool: 1
16 - outbox_missing_text: 2
17
18 ## Tasks
19
20 | Task | Category | Status | Steps | Tool Calls | Failure Types |
21 | --- | --- | --- | --- | --- | --- |
22 | chat_basic_math_no_tool | chat | PASS | 1 | 0 | - |
23 | chat_harness_short_explain | chat | FAIL | 1 | 0 | missing_expected_reply |
24 | chat_plain_style | chat | PASS | 1 | 0 | - |
25 | txt_attachment_summary | attachment | PASS | 2 | 1 | - |
26 | txt_attachment_grounded_question | attachment | PASS | 2 | 1 | - |
27 | txt_summary_to_md_outbox | attachment_output | PASS | 3 | 2 | - |
28 | txt_to_docx_report | docx | PASS | 3 | 2 | - |
29 | txt_to_pdf_report | pdf | FAIL | 4 | 3 | missing_expected_tool |
30 | xlsx_attachment_summary | xlsx | PASS | 2 | 1 | - |
31 | xlsx_filter_to_outbox | xlsx | FAIL | 4 | 2 | outbox_missing_text |
32 | xlsx_export_pdf_action | xlsx_action | PASS | 2 | 1 | - |
33 | xlsx_create_table_outbox | xlsx | PASS | 2 | 1 | - |
34 | docx_generate_project_brief | docx | PASS | 2 | 1 | - |
35 | docx_generate_checklist | docx | PASS | 2 | 1 | - |
36 | pdf_generate_project_brief | pdf | FAIL | 2 | 1 | outbox_missing_text |
37 | pdf_generate_runtime_gate_report | pdf | PASS | 2 | 1 | - |
38 | weather_script_query | weather | PASS | 2 | 1 | - |
39 | code_navigation_find_agent_loop | code_navigation | PASS | 5 | 4 | - |
40 | code_navigation_find_harness | code_navigation | PASS | 4 | 3 | - |
41 | outbox_list_current_session | tooling | PASS | 2 | 1 | - |
42
```

```
workspace > benchmarks > results > replay_latest.md > abc # MiniAgent Replay Report > abc ## Deterministic Replay
1  # MiniAgent Replay Report
2
3  - Run ID: 20260508_204222
4  - Source: workspace\traces\runtime_trace.jsonl
5  - Events: 50
6  - Roles: {}
7  - Kinds: {'memory_retrieval': 6, 'llm_request': 11, 'llm_response': 11,
8  'turn_completed': 6, 'skill_activation': 5, 'tool_call': 5, 'tool_result': 5,
9  'file_created': 1}
10 - Replay Status: passed
11 - Replayed Iterations: 11
12 - Replayed Tool Calls: 5
13 - Finish Reason: completed
```

5.4 Demo 截图





六、系统升级与扩展

6.1 可扩展架构

本项目采用模块化设计，因此具有较好的扩展能力。首先，在交互平台方面，当前系统以 QQBot 为主要入口，同时保留 CLI 作为测试入口。未来可以扩展到微信、飞书、Telegram、Discord 或 Web UI。由于系统内部使用统一消息格式，新增平台时只需要实现新的 Channel Adapter，而不需要修改 Agent 核心逻辑。

其次，在 Skill 能力方面，当前系统支持 PDF、Word、Excel、代码导航和天气查询等能力。未来可以继续增加 PPT 生成与修改、图片 OCR、LaTeX 编译、邮件处理、日历管理、数据库查询和数据可视化等 Skill。每个 Skill 可以继续采用 SKILL.md、actions.json 和 scripts 的组织方式接入系统。

再次，在工具系统方面，未来可以增加向量数据库检索工具、浏览器自动化工具、数据库连接工具、远程 API 调用工具、Python 数据分析工具和图像处理工具。这些工具可以统一注册到 ToolRegistry 中，由 Agent 根据任务需要调用。

此外，由于 QQBot 面向多用户场景，未来还需要进一步增强多用户隔离能力。系统可以按照 user_id 隔离会话、上传文件、输出文件和长期记忆，并加入文件访问权限控制，避免不同用户之间的数据混淆。

6.2 下一阶段计划

下一阶段可以从基础能力完善、多步任务规划、多 Agent 协作、安全性增强和评估体系完善几个方向继续改进。

首先，需要完善 QQBot Agent 的基础能力，包括增强 QQ 消息兼容性、优化文件上传处理、完善会话管理、提升基础问答体验和修复已知工具调用问题。

其次，未来可以支持更复杂的多步任务，例如“读取这个 Excel，统计销售额，生成图表，然后写入 Word 报告并导出 PDF”。这类任务需要系统将用户请求拆解为多个步骤，并按照依赖关系依次执行。

再次，可以引入多 Agent 协作机制。未来系统可以拆分为 Planner Agent、Executor Agent、Verifier Agent、Recovery Agent 和 Memory Agent。Planner Agent 负责任务拆解，Executor Agent 负责工具调用，Verifier Agent 负责结果检查，Recovery Agent 负责失败修复，Memory Agent 负责管理用户偏好和历史信息。

最后，需要增强系统安全性。由于 Agent 能够执行脚本和处理文件，未来应加入 Docker 沙箱、命令白名单、文件访问权限控制、脚本执行超时限制、网络访问限制和日志审计等机制，提高系统在真实部署场景中的安全性。

6.3 AI 能力演进路径

从 AI 能力演进角度看，QQBot 可以经历从命令式机器人到生产级智能 Agent 平台的多个阶段。第一阶段是命令式 QQBot，只能处理固定命令和规则回复。第二阶段是 LLM 问答型 QQBot，能够理解自然语言，但主要返回文本。第三阶段是工具调用型 QQBot Agent，能够根据任务调用工具，完成真实操作。第四阶段是多 Skill 任务型 Agent，能够处理 PDF、Word、Excel、代码搜索等复杂任务。第五阶段是多 Agent 协作系统，不同 Agent 分工完成规划、执行、验证和恢复。第六阶段是生产级智能 Agent 平台，支持多用户、多平台、多工具、安全隔离和持续评估。

本项目处于第三阶段到第四阶段之间，已经具备自然语言理解、工具调用、多 Skill 扩展、运行时检查和 Benchmark 评估能力。未来通过多步规划、多 Agent 协作和安全机制增强，可以进一步向生产级 Agent 平台演进。

七、课程总结

7.1 个人收获

通过这个项目，我最大的收获不是“做出了一个能聊天的助手”，而是开始真正理解 Agent 系统和普通 LLM 应用的区别。最开始我更容易把 Agent 理解成“模型 + prompt + tools”，但在实际开发中我逐渐意识到，真正困难的部分并不是把工具挂上去，而是如何让模型在真实任务里可靠地使用工具、基于真实证据作答、在失败时被发现并纠正。尤其是在文件处理任务中，模型很容易出现“语言上说自己做了，实际上并没有做”的假行为，例如没有读取文件却声称已检查内容、没有生成文件却声称已保存成功。这个项目让我认识到，Agent 开发本质上不是单纯追求“更强的推理能力”，而是要设计一套约束模型行为、验证执行结果、处理失败恢复的运行机制。

在 Agent 开发层面，我对 ReAct、Function Calling、Skill、Memory 之间的边界有了更清晰的理解。ReAct 提供的是“思考并调用工具”的基本循环，但它本身并不能保证任务一定完成；Function Calling 解决的是结构化调用接口问题，但不能自动解决真实性问题；Skill 更适合承载项目级能力包和脚本化操作，但 skill 被命中并不等于脚本真正执行；Memory 也不是简单地“把历史都塞给模型”，而是要区分短期上下文、长期事实和检索注入。我逐渐理解到，一个工程化 Agent

系统需要把这些能力拆开：模型负责选择和表达，runtime 负责约束和执行，memory 负责注入相关事实，skill 负责沉淀领域能力，评测系统负责告诉我们它到底有没有真的做成。

这个项目对我理解 Harness 的帮助尤其大。以前我对 harness 的理解更多停留在“跑 benchmark 的脚本”，但这次实现之后，我更倾向于把 harness 看成 Agent 的工程控制层。它不仅用于评测，更重要的是统一 live runtime、evaluation、trace、replay 和 regression 的执行路径，让线上行为和测试行为尽量一致。通过 RuntimeContext、AgentAssembly、AgentRuntimeSession、TraceSink 这些组件，我理解到 harness 的价值在于：它把原本分散在主程序、测试脚本和调试流程中的逻辑收拢成统一框架，使系统具备可观测、可回放、可比较、可诊断的能力。也正是因为有 harness，我才能比较清楚地定位问题到底发生在 skill 路由、tool 调用、文件证据、输出产物还是恢复策略上，而不是只能从最终回复里猜。

另一个很重要的收获是，我对“可验证性”在 Agent 系统中的地位有了更深的认识。以前我会比较自然地相信“只要 prompt 设计得更好，模型就会更听话”，但项目中的很多失败案例让我意识到，不能把系统可靠性建立在模型自觉之上。于是后面我把 TurnIntent、RuntimeVerifier、RuntimeRecovery 拆成独立模块，让 runtime 能先判断本轮是否需要文件证据、是否需要输出文件、是否需要脚本执行，再在最终回复前检查这些条件是否真的满足。如果不满足，就不允许直接结束，而是进入恢复或安全失败。这让我第一次真正感受到，Agent 工程的重点不只是“让模型会做事”，更是“让系统知道模型有没有真的做事”。

从工程实践角度看，这个项目也让我形成了更完整的开发闭环意识。过去写功能时，我更关注“能不能跑通”；现在我会更自然地考虑：这个行为有没有 trace，可不可以 replay，失败能不能复现，改完后有没有 regression 风险，评测数据能不能说明问题。这种从“功能开发”转向“运行时工程”和“可验证开发”的视角变化，是我在这个项目里最大的成长之一。

7.2 工程思维转变

这个项目带给我最明显的变化，是让我从“实现一个功能”逐渐转向“设计一套可运行、可验证、可维护的系统”。一开始我更关注的是模型能不能回答问题、能不能调用工具、能不能把任务跑通，但随着项目不断复杂，我逐渐意识到，真正的工程挑战并不在于让某个 demo 成功一次，而在于让系统在不同输入、不同渠道、不同任务场景下都能稳定工作，并且在出错时能够被发现、被定位、被修复。

这种转变最直接地体现在我对“完成任务”的理解上。以前我更容易把“模型说自己完成了”当作任务完成的标志，但在这个项目里，大量实际问题表明这远远不够。模型可能会假装读了文件、假装执行了脚本、假装保存了结果。于是我开始从“看回复是否合理”转向“看系统是否留下了真实证据”，也就是把判断标准从自然语言承诺转移到工具调用结果、输出产物、`trace` 记录和 `runtime` 校验上。这让我意识到，工程系统的可信性不能建立在模型自觉之上，而必须建立在结构化状态、可验证执行和明确控制逻辑之上。

我对系统分层的理解也因此更清晰。过去我会倾向于把很多逻辑直接塞进主流程里，比如通过 `prompt` 提醒模型“要先读文件”“要保存输出”“不要乱说”。但随着问题增多，我发现这种做法非常脆弱，也很难维护。后来我逐步把意图识别、工具协议、`skill` 路由、运行校验、失败恢复、`trace` 记录和 `benchmark` 评测拆开，让每一层只承担清晰的职责。这个过程让我真正体会到，工程设计不是把功能不断叠加，而是不断梳理边界、降低耦合、提升可观察性。

另一个重要变化是，我开始更重视“运行时”和“评测系统”本身，而不只是业务功能。以前我会把测试看成开发完成后的附属步骤，但在这个项目里，`harness`、`trace`、`replay`、`regression` 反而成了系统可靠性的重要组成部分。也正是在不断复盘失败案例的过程中，我理解到：一个成熟的 `Agent` 项目，不只是要有能力做事，还必须有能力证明它做了什么、为什么失败、改动后有没有变好。这种从“功能导向”转向“系统导向”和“证据导向”的思维变化，是我觉得这门课和这个项目带给我最深的成长。

7.3 对课程的建议

我认为课程对于 `Agentic AI` 的工程能力培养具有较强价值。课程不仅要求学生理解大语言模型的基本使用方式，还强调围绕具体应用场景设计一个完整 `Agent`，这使项目不再停留在简单调用 `LLM API` 的层面

同时，建议课程增加更多真实项目案例和失败案例分析。许多 `Agent` 项目在 `Demo` 中可以正常运行，但在真实任务中会遇到各种问题。如果课程能够结合具体案例讲解如何定位和改进这些问题，会更有助于学生形成工程化思维。除此之外，也可以在课程中加入 `AI IDE` 辅助开发、项目规格文档撰写、测试等内容，使学生不仅能够实现功能，还能够完成较规范的系统设计、开发和评估过程。